

Q. Spring boot

Q. What is spring boot?

Spring boot is project that is built on the top of spring framework. It provides easier and faster way to setup, configure and run both simple and web-based applications.

Means we can say spring boot is module of spring that provide Rapid application development feature and it is used for create stand alone spring based application and it need a minimal Configurations.

In Short we can say Spring boot is combination of

Spring Framework + Embedded HTTP -- Minimize XML configuration = spring boot.

servers

tomcat or jetty

Means spring boot contain embedded server means in spring boot there is no requirement for write the XML configuration everything manage by using annotation and all boilerplate configuration manage by spring boot internally means programmer not need to perform external configurations.

Q. Why use spring boot or what is benefit of spring boot?

1. The dependency injection approach is used in spring boot.
 2. It contains powerful database transaction management capabilities.
 3. It provide easier way to connect application with other framework like spring data JAP/hibernate etc.
 4. It reduces the cost of development.
 5. Spring boot some many sister concern framework or libraries to us like as
 1. spring data
 2. spring batch
 3. spring security
 4. spring integration etc.
 6. it contain embedded server
 7. It provide starter pom based maven configuration means provide some default libraries to us.
 8. There is a not required XML configuration
 9. Also minimum writing boiler plate configuration and code.
 10. Also spring boot production ready application with feature like as metrices, health checks and external configuration etc.
 11. Spring boot specially design work with REST APIS.
- Etc.

What is Spring boot?

Spring Boot is an open-source Java framework which provides a faster way to set up and an easier, configure, and run both simple and web-based applications. It is a combination of Spring Framework and Embedded Servers. The main goal of Spring Boot is to reduce development, unit test, and integration test time and in Spring Boot, there is no requirement for XML configuration.

Spring Boot is built on the top of the spring and contains all the features of spring. And is becoming favourite of developer's these days because of it's a rapid production-ready environment which enables the developers to directly focus on the logic instead of struggling with the configuration and set up. Spring Boot is a microservice-based framework and making a production-ready application in it takes very less time

Features of Spring Boot

Spring Boot is built on the top of the conventional spring framework. So, it provides all the features of spring and is yet easier to use than spring.

- **It allows to avoid heavy configuration of XML which is present in spring:** Unlike the [Spring MVC Project](#), in spring boot everything is auto-configured. We just need to use proper configuration for utilizing a particular functionality. For example: If we want to use [hibernate\(ORM\)](#) then we can just add @Table annotation above model/entity class(discussed later) and add @Column annotation to map it to table and columns in the database
- **It provides easy maintenance and creation of REST end points:** Creating a [REST API](#) is very easy in Spring Boot. Just the annotation @RestController and @RequestMapping(/endPoint) over the controller class does the work.
- **It includes embedded Tomcat-server:** Unlike [Spring MVC project](#) where we have to manually add and install the tomcat server, Spring Boot comes with an [embedded Tomcat server](#), so that the applications can be hosted on it.
- **Deployment is very easy, war and jar file can be easily deployed in the tomcat server:** war or jar files can be directly deployed on the Tomcat Server and Spring Boot provides the facility to convert our project into war or jar files. Also, the instance of Tomcat can be run on the cloud as well.
- **Microservice Based Architecture:** Microservice, as the name suggests is the name given to a module/service which focuses on a single type of feature, exposing an API(application peripheral interface). Let us consider an example of a hospital management system.
 - In case of monolithic systems, there will be a single code containing all the features which are very tough to maintain on a huge scale.
 - But in the microservice-based system, each feature can be divided into smaller subsystems like service to handle patient registration, service to handle database management, service to handle billing etc.

Microservice based system can be easily migrated as only some services need to be altered which also makes debugging and deployment easy. Also, each service can be integrated and can be made in different technologies suited to them.

Spring Boot Advantages

- It generates standalone Spring applications that are launched with the Java -jar command.
- It easily tests web applications using several Embedded HTTP servers like as Tomcat, Jetty, and others. We don't need to distribute WAR files.
- It provides 'starter' POMs with strong opinions to help us simplify our Maven settings.
- It includes features that are ready for production, such as metrics, health checks, and externalised settings.
- There is no need for XML configuration.
- It includes a command-line utility for designing and testing Spring Boot applications.
- It provides a number of plug-ins.
- It also minimizes writing multiple boilerplate codes, XML configuration, and annotations.
- It boosts productivity while decreasing development time.

Limitations of Spring Boot

While Spring Boot is powerful, it has some drawbacks to consider:

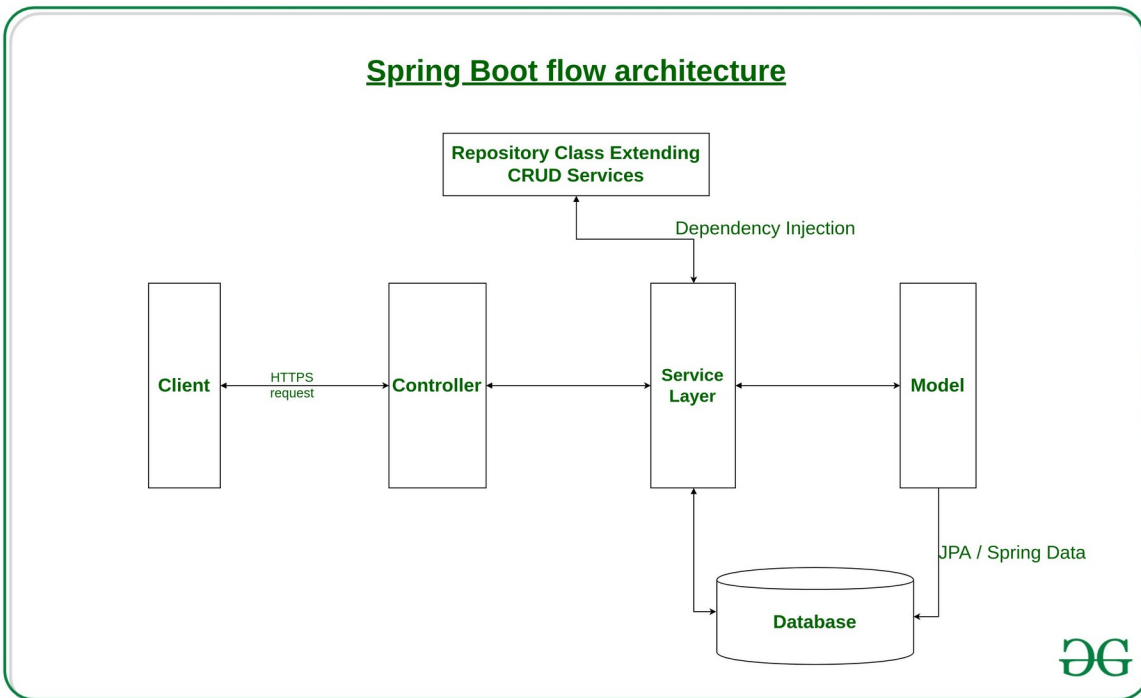
- It can be tricky to learn at first, especially for beginners.
- It may use more memory and take longer to start compared to simpler frameworks.
- Its conventions can restrict customization options for specific needs.
- Managing multiple microservices can get complicated.
- Upgrading or integrating with certain tools may be tricky.
- Applications built with Spring Boot can be larger to deploy.

Spring Boot Architecture

To understand the architecture of Spring Boot, let us first see different layers and classes present in it.

- **Layers in Spring Boot:** There are four main layers in Spring Boot:
 - **Presentation Layer:** As the name suggests, it consists of views(i.e. frontend part)
 - **Data Access Layer:** CRUD (create, retrieve, update, delete) operations on the database comes under this category.
 - **Service Layer:** This consists of service classes and uses services provided by data access layers.
 - **Integration Layer:** It consists of web different web services(any service available over the internet and uses [XML](#) messaging system).
- Then we have utility classes, validator classes and view classes.
- All the services provided by the classes are implemented in their corresponding classes and are retrieved by implementing the dependency on those interfaces.

Spring Boot flow architecture:



- Since Spring boot uses all the features/modules of spring-like Spring data, [Spring MVC](#) etc. so the architecture is almost the same as spring MVC, except for the fact that there is no need of **DAO** and **DAOImpl classes** in Spring boot.
- Creating a data access layer needs just a repository class instead which is implementing CRUD operation containing class.
- A client makes the https request(PUT/GET)
- Then it goes to controller and the controller mapped with that route as that of request handles it, and calls the service logic if required.
- Business logic is performed in the service layer which might be performing the logic on the data from the database which is mapped through JPA with model/entity class
- Finally, a JSP page is returned in the response if no error occurred.

If we want to work with spring boot we should have to know some important points before spring boot.

1. REST API:

Q. what is REST?

REST stands for Representational State Transfer basically it is built on http protocol and it is used for Accept any kind of request as well as return any kind of response because if we use REST API then we can send response in terms of string, object, JSON format etc

Note: refer JavaScript tutorial.

Q. What is the limitation of http protocol?

If we think about http protocol then the limitation is we can send request only in the form HTML page or we can response in the form html page.

The major limitation we can develop the web application using single technology

Example: suppose if we develop the website using http protocol in java then we can design view pages using .jsp and backend using java technology using servlet or spring mvc etc.

You cannot connect java server application with any other UI framework or mobile application means your java application cannot communicate with other application those design using different technology it is major limitation of http protocol

So avoiding these problems roy field introduce new concept which design with the help of http protocol or built on top up of http protocol just they modify the http protocol in terms of request and response Means plain http protocol accept request only in the form of html and get response in the form of html But RESTFUL Request can send response in the form objects, string, and get response in the form of objects or string etc.

If we think about restful web services we can transfer data as request or response using a following format.

- A) Using XML
- B) Using JSON

Now we want to discuss about JSON

JSON stands for java script object notations we can it is java script which contain data in the form of key and value pair and key present in the form of string and value may be any type

if we think about JSON it is language in dependent format means we can use JSON with any language like as we can use JSON in React,.net, java, php etc.

2. POSTMAN tool

POSTMAN is one the of the most popular software testing tool which is used for API Testing with the help of this tool developer can create , test and share APIS document.

Q. Why POSTMAN?

POSTMAN is desktop application or stand alone application platform to build, test, design, modify and document of API's. It is a simple graphic user interface for sending and viewing HTTP request and response.

POSTMAN provide GUI interface to us which help us to communicate with REST API and provide GUI interface for send form data or json data as request to server and get response from server and display on console application.

POSTMAN can make the various types of HTTP request like as GET, POST, PUT, DELETE, PATCH etc

How to use the POSTMAN Practically

If we want to work with POSTMAN practically we have to use following steps.

1. Download and Install POSTMAN

You can download The POSTMAN: <https://www.postman.com/downloads/>

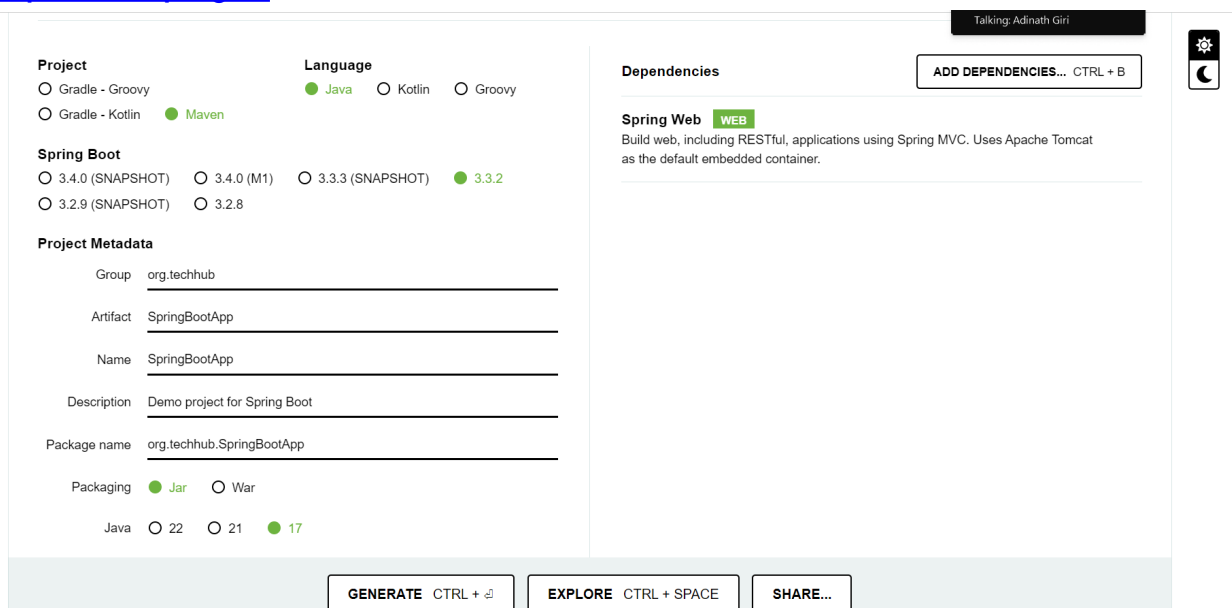
2. Create REST API Using Spring boot:

If we want to create REST API using spring boot we have to create application using spring boot.

How to create Application using spring boot

1. Open spring initializer website.

<https://start.spring.io/>



The screenshot shows the Spring Initializer web application. At the top right, it says "Talking: Adinath Giri". Below this, there are three main sections: "Project", "Language", and "Dependencies".

- Project:** Includes radio buttons for "Gradle - Groovy", "Gradle - Kotlin", and "Maven" (which is selected).
- Language:** Includes radio buttons for "Java" (selected), "Kotlin", and "Groovy".
- Spring Boot:** Includes radio buttons for "3.4.0 (SNAPSHOT)", "3.4.0 (M1)", "3.3.3 (SNAPSHOT)", and "3.3.2" (selected).
- Project Metadata:** Includes input fields for "Group" (org.techhub), "Artifact" (SpringBootApplication), "Name" (SpringBootApplication), "Description" (Demo project for Spring Boot), and "Package name" (org.techhub.SpringBootApplication).
- Packaging:** Includes radio buttons for "Jar" (selected) and "War".
- Java:** Includes radio buttons for "22", "21", and "17" (selected).
- Dependencies:** Includes a section for "Spring Web" with a "WEB" tag and a description: "Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container." There is also a button "ADD DEPENDENCIES... CTRL + B".

At the bottom, there are three buttons: "GENERATE CTRL + G", "EXPLORE CTRL + SPACE", and "SHARE...".

Note: create project and download it and open using any tool like as spring tool suite.

How to create application using spring boot

Note: Before that we should have to know how many types of application can create using spring boot

There are four categories of application can design by spring boot

1. Standalone application

Q. What is standalone application?

The application can be specific for one computer and allow using by only one user at a time called as standalone application.

Example: All core java application called as standalone application, desktop application called as standalone, desktop gaming application, calendar, calculator etc.

2. Web application: Normally we can say web application is client server architecture application where browser works as client software and interacts with server application or server side software called as web application.

Note: Web application for C2B Application means customer to business applications here browser works for customer and server works for business means when customer requires anything then customer can send request using browser and browser sends request to server and server can process and browser request and execute business logic and return response to customer on browser.

3. Distributed Application: distributed application allows different client applications to consume services of server application called as distributed application.

In the case of distributed application, client application is always some other software or application and communicates with a server application or another server application. It may be mobile application, browser, IOT Application or OTT Platform etc.

Normally Distributed application design for B2B Communication or business to business communication.

Examples of distributed application

Phone pay app

Example1

We can connect phone pay app for business transaction, we can connect phone from web application or we can connect phone pay application using scanner. Means we have a number of client applications that can communicate with server application for business transaction purpose called as distributed application.

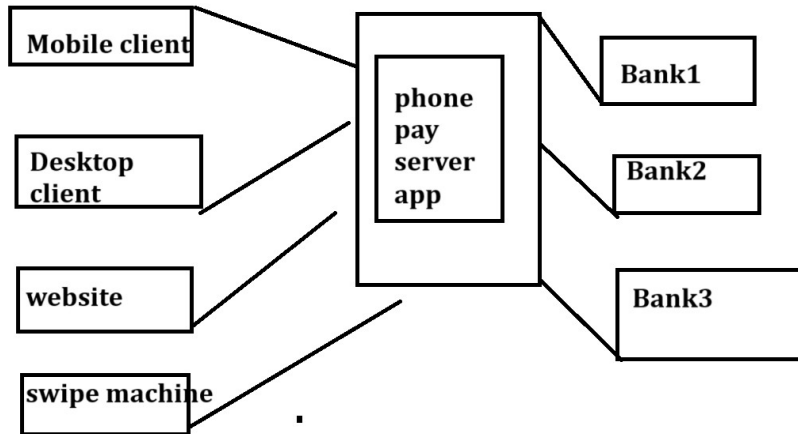
Example2: ICC live cricket score application

Suppose consider ICC public cricket score and Zee News show the update score to end users. Means Zee News designs own software which can communicate with ICC Application for business transaction. Here client application contains business logics and server application contains business logics so it is B2B communication. It is also distributed application.

Examples of Distributed Application

- a) UPA Payments (Google pay, phone pay etc)
- b) Payment Gateway APP (Visa, Master, Rupay)
- c) Payment broker application (Paypal, Razor Pay, nimo pay) etc.
- d) ICC (International Cricket Council)

E) NSE(Nation Stock exchange)
F) BSES (Bombay Stock Exchange)
etc



4. Enterprise level application:

Enterprise application is combination of web application + distributed application.

5. Micro services architectures

If we want to work with micro services architecture before that we should have to monolithic architecture.

Monolithic architecture:

If we think about monolithic architecture we have application with multiple services /modules and at the end of application development we bind the all modules in single war or jar called as monolithic architecture.

Some limitation of monolithic architecture

1. difficult to provide separate module to end user for single module required to provide complete project

Example: suppose consider we design banking application with four services like as account opening service, EMI calculator service, credit card and debit card issue service , loan processing service for particular bank with monolithic architecture so we combine all services in single war and provide to our customer but we have one customer he having own banking application with services account opening service, emi calculator service , credit card or debit card service just he required loan processing service from us then we cannot provide separate module as service to end user we need to provide all services either no service because loan module is part of whole project and it is dependent on some another module inputs so it is not possible.

2. Suppose we deploy application on server and if loan service use by millions of user then we need to upscale project on cloud but we need to upscale all modules because we have single war or single project on cloud so we need to up all project means all services so it may un necessary cost investment for other services which we not required to up

3. If we perform any change in any module then we need to rebuilt war for all modules and redeploy on server so it is tedious task in real time development

Etc.

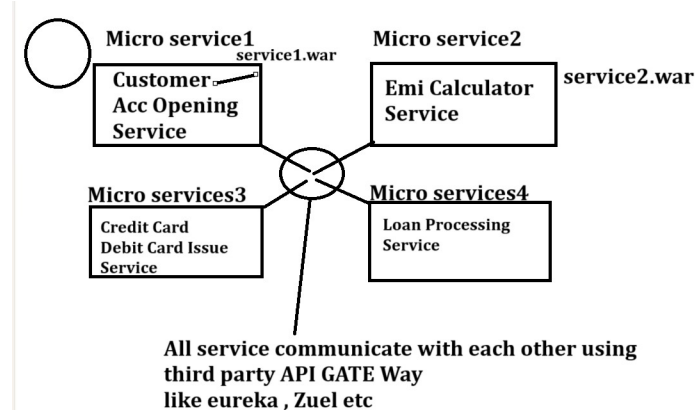
So over comes this problem micro service architecture comes in picture.

Q. What is meaning of micro services architecture and how it works?

If we think about micro services architecture here every service work as individual project or every module work as individual project and they can communicate internally with each other with the help of third Party API Gateway application.

Example: suppose if we take reference of previous example of monolithic architecture we have some different module like as loan service module, account opening module, EMI calculator module, Debit Card and Credit Card Issue Service then we can design separate project for every service with separate database and provide inter communication between all modules using some third party API Gate Way and using a standard design patterns

means if we think practically as per our example we have four micro services means four different spring boot project means four different war files which we upload on server



Note: we can design every micro service in different languages or using same language or tech stack.

Q. What is benefit of micro services?

1. We can use every service as individual application so we can integrate with any other application
2. We can deploy different kinds of application on cloud or server if any one service is down then complete application not down means your remaining services executing service provide service to end user
3. If particular service is majorly use by end users and if we want to up or provide more bandwidth to the service then we not need to upscale other services on server.
4. If we want to modify particular service then we not required to modify the code of other services and

As well as we can design every service in different tech stack so we get language independency for develop the applications

Note: Micro service architecture complete dependent on web services principal
Etc.

How to create application using spring boot

1. Installed Spring Tool Suite IDE & Create Project

file --- new --- spring starter project ---->give project name --- give group id and artifact id ----
select starter maven dependency (e.g spring web or spring data-jpa) --- click on next and click on finish

Q. what is starter dependency?

Starter maven dependency is a special type of dependency which help us to download the dependent jar as well as relative jar automatically so the benefit is programmer not need to add external or dependent jar manually.

Generalize format of starter dependency

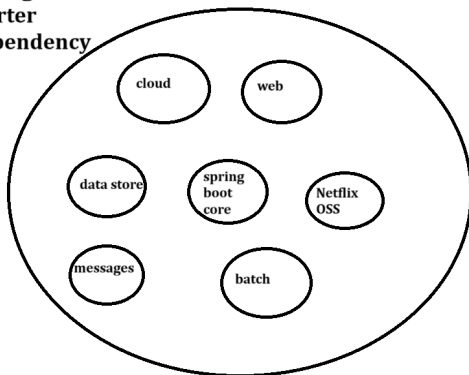
Syntax : spring-boot- starter-dependencyname

Example: spring-boot-starter-web
 spring-boot-starter-data-jpa

etc.

There are basic six types of starter dependency in spring boot

Spring boot
starter
dependency



cloud: this dependency helps to develop micro services architecture using boot.

web: this dependency help to develop web application as well as distributed application.

Netflix OSS: this dependency taking third party tool support while developing micro services architecture work as individual project and if you want to provide communication between them we need to support of third party tools so Netflix OSS comes in picture

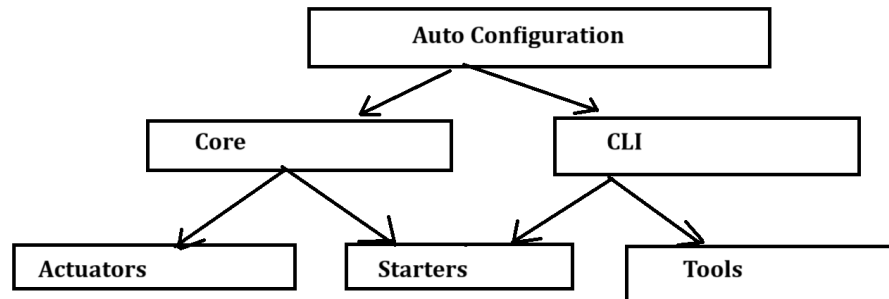
Note: Netflix is big organization that provides number of tools that required for inter communication between micro services based architecture.

Batch Starter: this starter helps us to batch processing technique.

Message Starters: this helps us to provide message based communications.

Data Source Starters: this starter helps us to provide database communication or interaction API to us

If we want to work with spring boot application we have some important components



Spring boot auto configuration feature based on jar spring boot can perform auto configuration automatically like as loading libraries class or readymade class and converting as bean Etc.

Spring boot core: this module provide supporting modules or it is core module of other modules means provide feature or core feature that required to other modules

Example: using command line argument and yaml or properties file spring environment property source automatically binding and apply validation on it.

CLI: Command Line interface based on ruby to start/stop spring boot created application test run and package the spring boot application.

Spring boot actuators: actuators enable certain enterprise features (security, metrics/default error) to your application as the auto configuration module it use auto detection to detect certain framework /feature of your application or we can say spring boot actuators are the nonfunctional feature will be added to every project to enable better monitoring and maintain the project.

Actuators provide the following feature to us.

1. Check the heap memory utilization
2. You can check the rest API using actuators
3. Taking the thread information, process information and taking care of heap memory information etc

Q. what is functional feature in project?

Functional feature means feature related to business logic of application called as functional feature.

Example: Student Registration, Teacher Registration, Batch Allocation to teacher etc

Q. What is nonfunctional feature in project?

Those feature not related to business logic of application called as nonfunctional feature.

Example: health info, thread info etc.

Once we create the spring boot project then we get one main class name given below.

```
package com.example.demo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringBootLastModuleApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringBootLastModuleApplication.class, args);
    }
}
```

If we think about above class it contain following things

@SpringBootApplication : it is summarize annotation of different types of annotation which help to enable auto configuration feature of spring boot as well as help us to configure bean classes and help us to scan the stereo annotation packages.

@SpringBootApplication annotation contain three types of annotations

@EnableAutoConfiguration: this annotation makes the spring boot framework to perform auto configuration i.e based on starters that added in class path it will give lots of common thing including the processing certain pre-defined classes as spring bean and doing injection on them

@Configuration and @SpringBootConfiguration: make the java classes as the spring bean configuration class where other java classes can be configured as spring beans it is alternative for xml file

@ComponentScan: component scan take base package as input and search stereo annotation and make then spring bean automatically means component scan search base packages as well as can search sub packages also

Q. What is stereo annotation?

Those annotation required for regular in application called as stereo annotation like as @Service, @Repository etc.

SpringApplication.run() : this method is used for provide spring IOC Container to us and get bean object using this IOC container means here we not need to create IOC container by externally using IOC Container API it is internally return by SpringApplication.run()

SpringApplication.run() method perform following task

- a. Bootstrap(start) the spring boot application
- b. Create IOC Container by taking current class name as the configuration class
- c. Based on starter the added and input that are given in the application.properties file perform auto configuration
- d. Perform component scanning to get stereo type annotation classes and make them as spring bean
- e. Automatically call bean methods to make the method return object as spring beans.
- f. Return ApplicationContext object as represent the IOC container

2. Create Rest Controller:

Q. What is RestController & benefit of REST ?

RestController help us to generate the REST API and the benefit of REST API is we can accept the request using any format and we can generate response using any format like as string,json,object et

Q. How to Create RestController using Spring boot?

if we want to create REST Controller using Spring boot we have annotation name as @RestController and it is combination of @Controller and @ResponseBody annotation
So if we want to Create REST Controller using Spring boot we have to create user define class under the source package and mark the user define with @RestController annotation

```
package com.example.demo.controller;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class TestCtrl {
}
```

Example: we want to design CRUD application Using Spring boot and use collection as temporary database.

The Major goal of task is

1. **Add new employee:** add new employee data in collection i.e. in ArrayList
2. **View employees:** view all employees from collection
3. **Search Employee:** Search employee from collection using its id.
4. **Delete employee:** delete employee from collection using its id
5. **Update Employee:** update employee data by using its id.

We want to design following REST Points or REST Services.

Rest point for save employee

REST Point	HTTP Method	Java method	Rest point descriptions.
/save	POST Annotation: @PostMapping	public String saveEmployee(@RequestBody Employee employee) Note: @RequestBody convert your json object in to plain java object means internally we required to json keys and model class variable name same for mapping purpose	Accept employee data and store in collection

@PostMapping or POST method can use for create new resource at server side

Example: save data at server side in database

Rest point for retrieve employee data

REST Point	HTTP Method	Java method	Rest point descriptions.
/view	GET Annotation: @GetMapping	public List<Employee> getAllEmployees() Note: this method can return all employee data from collection as response and convert java objects in to json format and convert list of java as array in javascript.	Retrieve all data from collections and display as response.

@GetMapping annotations: this annotation helps us to read data from server

Example: fetch all data of employee from a server, search data of employee by using its id.

Rest point for search employee data by using its id.

REST Point	HTTP Method	Java method	Rest point descriptions.
/search/{id}	GET Annotation: @GetMapping	public Employee getEmployeeById(@PathVariable("id") Integer id) Note: this method can accept id of employee and return data of employee. @PathVariable variable is annotation which is used for read data send by URL it is like as url rewriting in normal http request	Retrieve employee data from collection by matching its id and if id not found return then return null

Rest point for delete employee data by using its id.

REST Point	HTTP Method	Java method	Rest point descriptions.
/delemp/{id}	Annotation: @DeleteMapping	public String delEmp(@PathVariable("id") Integer id) Note: this method can accept id of	Search data using id and delete from collection if record not found return -1 and show message

		employee using path variable and if id found then delete record of employee and show message employee deleted successfully otherwise show message employee not deleted.	employee data not found.
--	--	---	--------------------------

@Delete mapping annotation help us perform deletion or delete resource at server side.

Rest Point for update employee data.

REST Point	HTTP Method	Java method	Rest point descriptions.
/updemp/	Annotation: @UpdateMapping	public String updateEmp(@RequestBody Employee emp) Note: this method can accept the employee data and update it using its id.	Search data by using its id and update it.

Steps to develop the above applications.

1. Create spring boot project using spring initilizr

<https://start.spring.io/>

Example:

```
package org.techhub.SpringBootFirstCRUDAPP.controller;
```

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.bind.annotation.RestController;
import org.techhub.SpringBootFirstCRUDAPP.model.Employee;
```

```
import java.util.*;
@RestController
public class TestCtrl {
    List<Employee> list=new ArrayList<Employee>();
    @PostMapping("/save")
    public String saveEmployee(@RequestBody Employee employee) {
```

```

        boolean b=list.add(employee);
        if(b) {
            return "Employee Added Successfully.....";
        }
        else {
            return "Employee not added.....";
        }
    }

}

@GetMapping("/view")
public List<Employee> getAllEmployees(){
    return list;
}

@GetMapping("/search/{id}")
public Employee getEmployeeById(@PathVariable("id") Integer id) {
    Employee emp=null;
    for(Employee e:list) {
        if(e.getId()==id) {
            emp=e;
        }
    }
    return emp!=null?emp:null;
}

@DeleteMapping("/delEmp/{id}")
public String delEmp(@PathVariable("id") Integer id) {
    System.out.println("Employee id "+id);
    int index=-1;
    for(Employee e:list) {
        if(e.getId()==id) {
            index=list.indexOf(e);
            if(index!=-1) {
                list.remove(index);
            }
        }
    }
    return index!=-1?"Employee Deleted":"Employee not found";
}

@PutMapping("/updateemp")
public String updateEmployee(@RequestBody Employee employee) {
    int index=-1;
    for(Employee e:list) {

```



```

        if(e.getId()==employee.getId()) {
            index=list.indexOf(e);
            if(index!=-1) {
                list.set(index, employee);
            }
        }
    }
    return index!=-1?"Employee update":"Employee not found";
}
}

```

How to Connect Spring boot application by using JPA

Steps.

1. Add spring data jpa maven dependencies & mysql connector maven dependencies.

```

<!-- https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-data-jpa -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
    <version>3.3.2</version>
</dependency>

```

```

<!-- https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
</dependency>

```

2. Open the application.properties file and write the following configurations from src/main/resources folder

application.properties

```

spring.application.name=SpringBootFirstCRUDAPP
spring.datasource.url=jdbc:mysql://localhost:3306/mysql
spring.datasource.username=root
spring.datasource.password=root

```

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

3. Create Entity class and map with database table

```
package org.techhub.SpringBootFirstCRUDAPP.model;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.Id;
import jakarta.persistence.Table;
@Entity
@Table(name = "springjulyemp")
public class Employee {
    @Id
    @Column(name = "empid")
    private int id;
    @Column(name = "empname")
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
    @Column(name = "empsal")
    private int sal;
}
```

4. Create Repository by extending JpaRepository

If we want to create the repository using spring boot we have to create user defined interface and we required to inherit the JpaRepository in it.

Example with source code

```
package org.techhub.SpringBootFirstCRUDAPP.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import org.techhub.SpringBootFirstCRUDAPP.model.Employee;
public interface EmployeeRepository extends JpaRepository<Employee, Integer>
{
}
```

Q. what is JpaRepository?

It is inbuilt interface and child of CrudRepository which provide some inbuilt method to us which help us to perform basic database operation like as save record in database, delete record from database ,search record in database etc.

S save(S entity): this method can accept entity class object and store database table
Means this can store your java object as row in database table.

Optional<T> findById(ID id): this method can search record from database table using its id or using primary key column database table.

boolean existsById(ID id): this method can search record in database table using its id or primary key and if record present return true otherwise return false.

Iterable<T> findAll(): this method can fetch all record or row as object in java from database table.

long count(): this method can give number of element count.

void deleteById(ID id): this method can delete record by using its id.

void delete(T entity): this method can delete record by entity object.

Following code show the repository interface using spring boot

```
package org.techhub.SpringBootFirstCRUDAPP.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import org.techhub.SpringBootFirstCRUDAPP.model.Employee;
@Repository("empRepo")
public interface EmployeeRepository extends JpaRepository<Employee, Integer>
```

```
{  
}
```

5. Create Service layer and call JpaRepository methods by using user defined interface

EmployeeService.java

```
package org.techhub.SpringBootFirstCRUDAPP.service;  
import java.util.Iterator;  
import java.util.List;  
import java.util.Optional;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Controller;  
import org.springframework.stereotype.Service;  
import org.techhub.SpringBootFirstCRUDAPP.model.Employee;  
import org.techhub.SpringBootFirstCRUDAPP.repository.EmployeeRepository;  
@Service("empService")  
public class EmployeeService {  
    @Autowired  
    EmployeeRepository empRepo;  
    public boolean isAddEmployee(Employee emp) {  
        Employee e = empRepo.save(emp);  
        return e != null ? true : false;  
    }  
    public List<Employee> getAllEmployees() {  
        List<Employee> list=empRepo.findAll();  
        return list;  
    }  
    public Employee getEmployeeById(int id) {  
        Optional<Employee> o=empRepo.findById(id);  
        return o.isEmpty()?null:o.get();  
    }  
    public String deleteEmployeeById(int id) {  
        Optional<Employee> o=empRepo.findById(id);  
        if(o.isEmpty()) {  
            return "Employee not present";  
        }  
        else {  
            empRepo.deleteById(id);  
            return "Employee Deleted Successfully....";  
        }  
    }  
}
```

```

}
public String updateEmployee(Employee emp) {
    Employee e=empRepo.save(emp);
    return e!=null?"Employee updated":"Employee not updated";
}
}

```

6. Create RestController and Create Rest End points and call service methods

```

package org.techhub.SpringBootFirstCRUDAPP.controller;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.bind.annotation.RestController;
import org.techhub.SpringBootFirstCRUDAPP.model.Employee;
import org.techhub.SpringBootFirstCRUDAPP.service.EmployeeService;
import java.util.*;
@RestController
public class TestCtrl {
    @Autowired
    EmployeeService empService;
    List<Employee> list = new ArrayList<Employee>();
    @PostMapping("/save")
    public String saveEmployee(@RequestBody Employee employee) {
        boolean b = empService.isAddEmployee(employee);
        if (b) {
            return "Employee Added Successfully.....";
        } else {
            return "Employee not added.....";
        }
    }

    @GetMapping("/view")
    public List<Employee> getAllEmployees() {
        return empService.getAllEmployees();
    }
}

```

```

@GetMapping("/search/{id}")
public String getEmployeeById(@PathVariable("id") Integer id) {
    Employee e = empService.getEmployeeById(id);
    return e != null ? e.toString() : "Employee not found";
}

@DeleteMapping("/delEmp/{id}")
public String delEmp(@PathVariable("id") Integer id) {
    String s = empService.deleteEmployeeById(id);
    return s;
}

@PutMapping("/updateemp")
public String updateEmployee(@RequestBody Employee employee) {
    String s=empService.updateEmployee(employee);
    return s;
}
}

```

Now we want to connect above application with react

Q. What is axios Library?

React cannot support directly to http protocols and if we want to send request or get response using http protocol in React application we required to use third party library known as axios library means we can say here axios library help us to consume the restful services in react.

How to send http status code as response from REST API to client side application

Http status code is used for send the message from server to browser to indicate whether or not that request fulfilled or not.

All status code set by W3C and the status code embedded in HTTP header of the page tell browser the result of its request.

Suppose consider we are working on Employee Management Application and we are using spring boot and we have some REST API with valid HTTP Protocol.

HTTP Method	URI	Description	Validate Http Status code
POST	saveEmployee	Create New Employee	201
GET	searchEmployee/{eid}	Read employee using id	200 if employee not found then return code 204
DELETE	deleteEmployeeById/{empid}	DELETE employee using id	204
GET	viewallemp	Get all employees	200,204

If we want to send Http status code using spring boot we have class name as ResponseEntity and HttpStatus

Syntax:

```
ResponseEntity<T> ref=ResponseEntity.status(HttpStatus.statuscode).build();
```

Sample code is given below

```
package com.example.demo.controller;
```

```
import org.slf4j.LoggerFactory;
import org.slf4j.Logger;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity.BodyBuilder;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
```

```

import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import com.example.demo.entity.Employee;
import com.example.demo.service.EmployeeService;
import java.util.*;
@RestController
public class EmployeeController {
    Logger logger = LoggerFactory.getLogger(EmployeeController.class);
    @Autowired
    EmployeeService empService;
    @PostMapping("/saveEmployee")
    public ResponseEntity<Employee> createEmployee(@RequestBody Employee employee) {

        Employee e=empService.saveEmployee(employee);
        if(e!=null) {
            ResponseEntity<Employee>
entity=ResponseEntity.status(HttpStatus.CREATED).build();
            return entity;
        }
        else {
            ResponseEntity<Employee>
entity=ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
            return entity;
        }

    }
    @GetMapping("/viewallemp")
    public ResponseEntity<List<Employee>>getAllEmployees(){
        List<Employee> list=empService.getAllEmployees();

        if(list.size()<=0) {
            /*ResponseEntity<List<Employee>>
entity=ResponseEntity.status(HttpStatus.NOT_FOUND).build();
            return entity;*/
            /*
            * or
            ResponseEntity.BodyBuilder build=ResponseEntity.status(HttpStatus.NOT_FOUND);
            ResponseEntity<List<Employee>> entity=build.build();

            return entity;*/
            //or

```



```

        return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
    }
    else {
        ResponseEntity<List<Employee>> entity=ResponseEntity.of(Optional.of(list));
        return entity;
    }
}

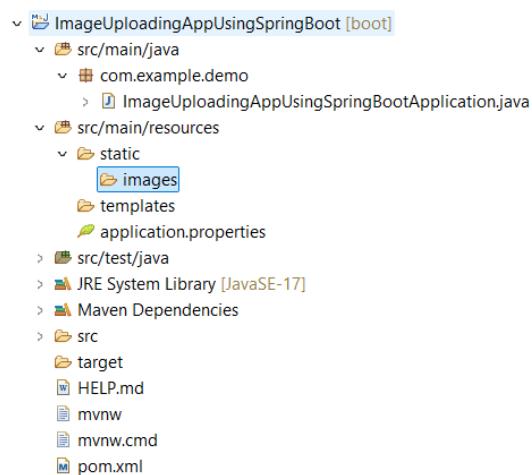
@DeleteMapping("deleteEmployeeById/{empid}")
public String deleteEmployeeById(@PathVariable("empid") Integer id) {
    String str=empService.deleteRecordById(id);
    return str;
}
}

```

How to upload the image at server side using spring boot

If we want to upload image at server side we have some following steps.

1. Create Spring boot application
2. Create static folder under resources folder.



3. Open the application.properties or application.yaml write following configuration related with file uploading.

```

spring.application.name=ImageUploadingAppUsingSpringBoot
spring.servlet.multipart.enabled=true
spring.servlet.multipart.max-file-size=300mb
spring.servlet.multipart.file-size-threshold=2kb

```

4. Create REST Controller and write the following code.

FileUploadingController.java

```
package com.example.demo.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;

import com.example.demo.entity.Student;
import com.example.demo.helper.ImageHelper;
import com.example.demo.service.StudentService;

@RestController
public class FileUploadingController {

    @Autowired
    StudentService studService;
    @Autowired
    ImageHelper imageHelper;
    @PostMapping("/saveStudent")
    public String createNewStudent(@RequestParam("image") MultipartFile image){
        boolean b=imageHelper.isSaveImage(image);
        return b?"Image uploaded ":"image not uploaded";
    }

}
```

ImageHelper.java

```
package com.example.demo.helper;

import java.io.FileOutputStream;
import java.io.InputStream;
import org.springframework.stereotype.Component;
import org.springframework.web.multipart.MultipartFile;
@Component("imageHelper") // @Bean(name="imageHelper")
```

```

public class ImageHelper {
    public static final String rootPath="D:\\SpringBootReactApp"
        + "\\ImageUploadingAppUsingSpringBoot\\src\\main\\resources\\static";
    public boolean isSaveImage(MultipartFile image) {
        try {
            InputStream input=image.getInputStream();
            byte b[]=new byte[input.available()];
            input.read(b);
            FileOutputStream fout=new
FileOutputStream(rootPath+"\\ "+image.getOriginalFilename());
            fout.write(b);
            fout.close();
            return true;

        }
        catch(Exception ex) {
            return false;
        }
    }
}

```

How to write Custom Exception or User Defined Exception in spring boot and handle it

Q. what is user defined exception?

Those exception define by programmer called as user defined exceptions

Q. Why we need to handle the user defined exceptions in spring boot?

There are two reasons to handle the user defined exceptions

1. When developer want to generate exception message according to project requirement
2. When Developer has some Exception but java not provide inbuilt class or API for that then developer can write own exception class called as User Defined Exceptions.

If we want to user defined exception using spring boot we have two ways

1. You can handle exception at controller level
2. You can handle at global level.

If we want to handle the user defined exceptions using spring boot we have to use two annotations

@ExceptionHandler : this annotation help us to handle the user defined exception at controller level.

@ControllerAdvice or **@RestControllerAdvice**

these annotations help us to handle the exception at global level or for whole application when we have `@Controller` then we can use `@ControllerAdvice` annotation and when we have `RestController` then we can use `@RestControllerAdvice` annotations.

Steps to handle the user defined exception using spring boot

1. Create Spring boot project

2. Create User Defined Exception class

```
package com.example.demo.customexe;
public class VoterException extends RuntimeException {
    private String message;
    public VoterException(String message) {
        this.message=message;
    }
    public String getMessage() {
        return message;
    }
}
```

3. Create One StandardError Message class

Normally using this class we send standard error message with standard http status code.

```
package com.example.demo.customexe;
public class ErrorMessage {
    private int statusCode;
    private String errorMessage;
    public int getStatusCode() {
        return statusCode;
    }
    public void setStatusCode(int statusCode) {
        this.statusCode = statusCode;
    }
    public String getErrorMessage() {
        return errorMessage;
    }
    public void setErrorMessage(String errorMessage) {
        this.errorMessage = errorMessage;
    }
    public ErrorMessage(int statusCode, String errorMessage) {
        this.statusCode = statusCode;
    }
}
```

```
this.errorMessage = errorMessage;
}
}
```

4. Create Model class

```
package com.example.demo.model;
public class Voter {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    private int age;
}
```

5. Create Service class

```
package com.example.demo.service;
import org.springframework.stereotype.Service;
import com.example.demo.customexe.VoterException;
import com.example.demo.model.Voter;
@Service("voterService")
public class VoterService {
    public String saveVoter(Voter voter) {
        if(voter.getAge()>18) {
            return "Voter Added in xbase";
        }
        else {
```

```
throw new VoterException("Invalid voter");
}
}
}
```

6. Create RestControllerAdvice class for mark as global exception class

```
package com.example.demo.customexe;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;
@RestControllerAdvice
public class GlobalException {
    @ExceptionHandler(value=VoterException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorMessage handleException(VoterException ex){
        return new ErrorMessage(HttpStatus.NOT_FOUND.value(), ex.getMessage());
    }
}
```

7. Create Rest Controller

```
package com.example.demo.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import com.example.demo.customexe.ErrorMessage;
import com.example.demo.customexe.VoterException;
import com.example.demo.model.Voter;
import com.example.demo.service.VoterService;

@RestController
public class VoterController {
    @Autowired
    VoterService voterService;
```

```
    @PostMapping("/saveVoter")  
    public String saveVoter(@RequestBody Voter voter) {  
        return voterService.saveVoter(voter);  
    }  
}
```